
MOTOR DE INFERENCIA LÓGICA COMO SERVICIO

Documentación Técnica del Proyecto

Materia: Teoría de Lenguajes de Programación

Fecha: Mayo 2025

1. Instrucciones de Instalación Local

A continuación se describen los pasos necesarios para instalar y ejecutar el servicio en un entorno local.

1.1 Requisitos Previos

Antes de comenzar, asegúrese de contar con el siguiente software instalado en su sistema:

- **Node.js v18** o superior — <https://nodejs.org>
- **npm v9** o superior (incluido con Node.js)
- **Git** (opcional, para clonar el repositorio)
- **Terminal o línea de comandos** (PowerShell, Bash, Zsh)

1.2 Obtención del Proyecto

Descargue o clone el repositorio del proyecto:

```
git clone <url-del-repositorio>
cd motor-inferencia-logica
```

O bien, si recibió el proyecto como archivo comprimido, extráigalo y navegue a la carpeta raíz del proyecto.

1.3 Instalación de Dependencias

Ejecute el siguiente comando en la raíz del proyecto para instalar todas las dependencias necesarias:

```
npm install
```

Esto instalará automáticamente las siguientes bibliotecas definidas en `package.json`:

Paquete	Versión	Descripción
express	<code>^4.18.2</code>	Framework HTTP para el servidor REST
tau-prolog	<code>^0.3.2</code>	Intérprete de Prolog en JavaScript

1.4 Estructura del Proyecto

Una vez instaladas las dependencias, el proyecto tendrá la siguiente estructura:

```
motor-inferencia-logica/  
├─ index.js           <- Servidor Express + Motor de inferencia  
├─ knowledge_base.pl  <- Base de conocimiento en Prolog  
├─ package.json       <- Configuración del proyecto y dependencias  
└─ node_modules/      <- Dependencias instaladas (generado por npm)
```

2. Instrucciones de Ejecución

2.1 Iniciar el Servidor

Para iniciar el servidor en modo producción, ejecute:

```
npm start
```

Para desarrollo con reinicio automático al guardar cambios (Node.js v18+):

```
npm run dev
```

Una vez iniciado, el servidor mostrará en consola:

 Motor de Inferencia ejecutándose en `http://localhost:3000`

2.2 Verificar el Estado del Servicio

Abra un navegador o cliente HTTP (como Postman) y visite:

```
GET http://localhost:3000/health
```

Respuesta esperada:

```
{
  "status": "ok",
  "service": "Motor de Inferencia Lógica",
  "engine": "Tau Prolog"
}
```

2.3 Consultar la Base de Conocimiento

El endpoint principal acepta consultas Prolog mediante POST:

```
POST http://localhost:3000/query
Content-Type: application/json
```

Cuerpo de la petición:

```
{
  "query": "penalty_applicable(X)."
}
```

Respuesta del motor de inferencia:

```
{
  "status": "ok",
  "query": "penalty_applicable(X).",
  "satisfiable": true,
  "count": 2,
  "answers": [
    { "X": "contract1" },
    { "X": "contract3" }
  ]
}
```

2.4 Ejemplos de Consultas Disponibles

Las siguientes consultas pueden ejecutarse sobre la base de conocimiento incluida:

Consulta	Descripción
<code>penalty_applicable(contract1).</code>	¿Se aplica penalización a contract1?
<code>penalty_applicable(X).</code>	¿A cuáles contratos aplica penalización?
<code>at_risk(X).</code>	Contratos activos con incumplimiento
<code>valid_contract(X).</code>	Contratos activos sin incumplimiento
<code>client_penalized(X).</code>	Clientes con penalización aplicable
<code>applicable_penalty_amount(X, Y).</code>	Contrato X tiene penalización de \$Y
<code>same_type(contract1, X).</code>	Contratos del mismo tipo que contract1

2.5 Otros Endpoints

- `GET /` — Documentación de la API con ejemplos de consultas
- `GET /facts` — Lista todos los hechos cargados en la base de conocimiento
- `GET /health` — Verificación del estado del servicio
- `POST /query` — Ejecuta una consulta Prolog (endpoint principal)

3. Paradigmas de Programación Utilizados

El proyecto integra tres paradigmas de programación fundamentales, cada uno cumpliendo un rol específico en la arquitectura del sistema.

3.1 Programación Lógica (Prolog / Tau Prolog)

El paradigma de programación lógica es el núcleo del sistema. En lugar de describir cómo resolver un problema paso a paso, se declaran hechos y reglas que describen las relaciones del dominio, y el motor de inferencia determina automáticamente las conclusiones.

Rol en el sistema

La base de conocimiento (`knowledge_base.pl`) contiene los hechos y reglas del dominio de contratos y penalizaciones. Tau Prolog actúa como el motor de inferencia que evalúa las consultas aplicando unificación y backtracking automático.

Características clave aplicadas

- **Hechos:** Verdades simples del dominio, como `expired(contract1)` o `has_penalty_clause(contract1)`.
- **Reglas:** Conclusiones derivadas mediante implicación lógica (`:-`), como `penalty_applicable(Contract) :- expired(Contract), has_penalty_clause(Contract), breach_reported(Contract)`.
- **Unificación:** El motor liga variables lógicas (`X`) automáticamente al evaluar consultas como `penalty_applicable(X)`.
- **Backtracking:** El motor explora todas las soluciones posibles de manera automática, retornando múltiples resultados cuando existen.
- **Negación por falla (`\+`):** Usada en `valid_contract` para verificar la ausencia de incumplimientos.

Ejemplo en el código:

```
% Regla compuesta con múltiples condiciones
penalty_applicable(Contract) :-
    contract(Contract),
    expired(Contract),
    has_penalty_clause(Contract),
    breach_reported(Contract).
```

3.2 Programación Funcional (JavaScript)

El paradigma funcional se aplica en la capa de transformación de datos del servidor Node.js. Se privilegia el uso de funciones puras, inmutabilidad y composición para el manejo de la entrada y salida del sistema.

Rol en el sistema

Las funciones funcionales se encargan de normalizar las consultas recibidas, formatear las respuestas JSON y convertir los términos Prolog a estructuras JavaScript legibles.

Características clave aplicadas

- **Funciones puras:** `normalizeQuery()`, `buildSuccessResponse()` y `buildErrorResponse()` no producen efectos secundarios y siempre retornan el mismo resultado para los mismos argumentos.
- **Transformación con map/filter:** Usados para iterar sobre los bindings Prolog y sobre las líneas de la base de conocimiento en el endpoint `/facts`.
- **Funciones de orden superior:** `termToJS()` aplica recursión funcional para transformar árboles de términos Prolog.
- **Spread operator y desestructuración:** Para construir objetos de respuesta de forma declarativa e inmutable.

Ejemplo en el código:

```
// Función pura: normaliza la consulta sin efectos secundarios
const normalizeQuery = (raw) => {
  const trimmed = String(raw).trim();
  return trimmed.endsWith('.') ? trimmed : `${trimmed}.`;
};
```

3.3 Programación Asíncrona (Node.js / async-await)

El modelo de ejecución de Node.js, basado en un event loop de un solo hilo con I/O no bloqueante, es aprovechado para manejar múltiples solicitudes concurrentes sin bloquear el servidor.

Rol en el sistema

Las operaciones de lectura del archivo de base de conocimiento y la ejecución del motor Prolog (que opera mediante callbacks internos) se envuelven en Promises y se consumen mediante `async/await`, permitiendo código asíncrono legible y mantenible.

Características clave aplicadas

- **async/await:** El handler del endpoint `/query` es una función `async` que aguarda la lectura del archivo y la resolución de la inferencia.
- **Promises:** `runPrologQuery()` encapsula la API de callbacks de Tau Prolog en una Promise, normalizando la interfaz asíncrona.
- **fs.promises:** Lectura no bloqueante del archivo `.pl` mediante `fs.readFile` (API de Promesas de Node.js).
- **Recursión asíncrona:** `getNextAnswer()` se llama recursivamente para iterar sobre las soluciones de Prolog, respetando el modelo de callbacks del motor.
- **Concurrencia:** Node.js puede atender múltiples solicitudes simultáneas gracias al event loop, sin necesidad de hilos adicionales.

Ejemplo en el código:

```
app.post('/query', async (req, res) => {
  // Operación asíncrona de I/O
  const kbSource = await fs.readFile(KB_PATH, 'utf8');

  // Operación asíncrona de inferencia lógica
  const answers = await runPrologQuery(kbSource, normalizedQuery,
maxSols);

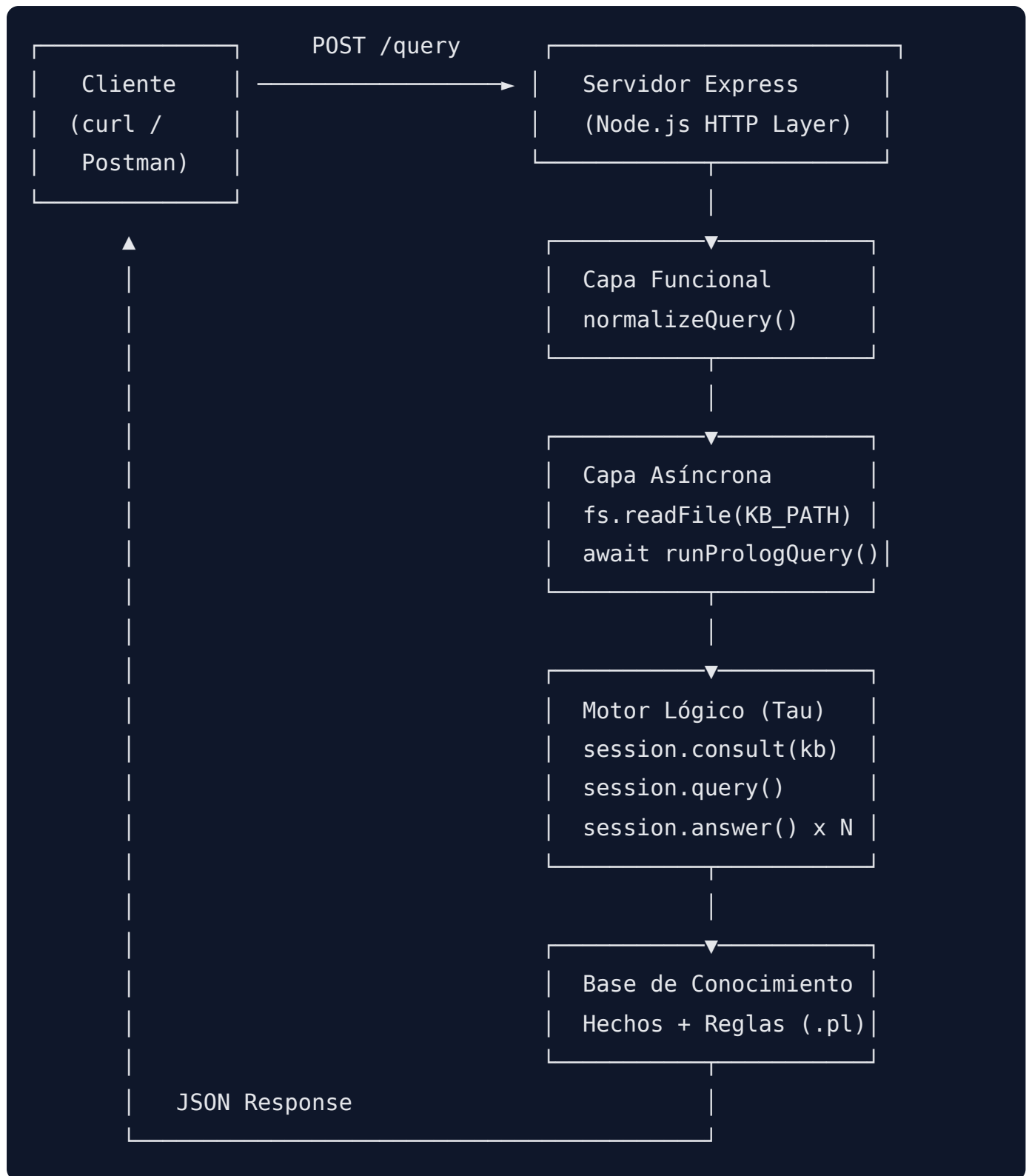
  return res.json(buildSuccessResponse(normalizedQuery, answers));
});
```

4. Arquitectura del Sistema

El sistema sigue una arquitectura de servicio de un solo módulo donde los tres paradigmas de programación colaboran en una cadena de procesamiento bien definida.

4.1 Diagrama de Flujo de una Solicitud

El siguiente diagrama describe el ciclo de vida de una consulta desde que el cliente la envía hasta que recibe la respuesta:



4.2 Componentes del Sistema

Componente	Tecnología	Responsabilidad
Servidor HTTP	Express.js	Recibir y enrutar solicitudes REST, enviar respuestas JSON

Capa Funcional	JavaScript (ES6+)	Normalización de entrada, formateo de salida, transformación de datos
Motor de Inferencia	Tau Prolog	Evaluar consultas lógicas mediante unificación y backtracking
Base de Conocimiento	Prolog (.pl)	Almacenar hechos y reglas del dominio de contratos
Capa Asíncrona	async/await + Promises	Manejar I/O no bloqueante y ejecución concurrente

4.3 Interacción entre Paradigmas

La arquitectura del sistema aprovecha las fortalezas de cada paradigma de forma complementaria:

- **El paradigma asíncrono** maneja la capa de transporte: recepción de solicitudes HTTP concurrentes, lectura del archivo de la base de conocimiento y entrega de respuestas sin bloquear el event loop.
- **El paradigma funcional** actúa como capa de transformación: funciones puras convierten la entrada del cliente a un formato adecuado para el motor lógico, y la salida del motor a un JSON estructurado para el cliente.
- **El paradigma lógico** implementa el dominio del negocio: toda la lógica de contratos, penalizaciones y reglas de inferencia se expresa de forma declarativa en Prolog, separando el "qué" del "cómo".

4.4 Decisiones de Diseño

- **Sin base de datos externa:** La base de conocimiento es un archivo `.pl` cargado en memoria por cada consulta, lo que simplifica el despliegue y facilita la comprensión del paradigma lógico.
- **Stateless por solicitud:** Cada petición crea su propia sesión Prolog, garantizando aislamiento y evitando efectos secundarios entre solicitudes.
- **Máximo de soluciones configurable:** El parámetro `max_solutions` permite al cliente controlar el límite de respuestas, evitando la generación infinita en consultas muy generales.

- **Manejo de errores por capas:** Los errores de Prolog (sintaxis, inferencia) se distinguen de los errores de I/O, retornando mensajes descriptivos al cliente.